

Simple Incident Prediction with Sliding Windows

Idea

The task is to predict if an incident will happen within the next H time steps based on the previous W time steps of a metric. I treated this as a simple binary classification problem:

- 0 = no incident soon
- 1 = incident will happen soon

The model just looks at recent metric values and tries to decide if an incident is coming in the near future.

Instead of using a real dataset, I generated a synthetic metric because the task allowed that and it keeps the project simple.

Data Generation

I simulate a metric that slowly moves around a baseline value (around 50). Each step I add small Gaussian noise so the metric changes gradually.

Sometimes I trigger an incident. When that happens, I add a large spike to the metric and the incident persists for multiple time steps (5-11 steps). This makes incidents more predictable since the metric stays elevated. The incident probability was set to 8% with spike magnitudes between +20 and +35.

So in the end I get two arrays:

- metric values over time
- incident labels (0 or 1)

Incidents correspond to sustained elevated periods in the metric.

Sliding Windows

Machine learning models need fixed-size inputs, so I convert the time series into sliding windows.

Example:

- window size $W = 5$
- prediction horizon $H = 2$

Metric:

[50.1, 49.9, 50.2, 50.0, 50.4, 70.0, 71.3]

One training sample becomes:

[50.1, 49.9, 50.2, 50.0, 50.4] $\rightarrow$ 1

Why 1? Because an incident happens within the next 2 steps (the spike).

The window then slides forward by one step and the process repeats. This produces many training samples where:

- X = past metric values + derived features (mean, std, max, min, trend)
- y = whether an incident happens soon

Model Choice

I used logistic regression from scikit-learn. This is a simple binary classification model and is more than enough for a minimal solution.

Some small things I added:

- **StandardScaler** to normalize the features. Normalization ensures that all metric values are on a similar scale, which helps the optimizer converge faster and avoids features with larger ranges dominating the learning process.
- **class_weight="balanced"** because incidents are rare. Since only a small fraction of samples correspond to incidents, this tells the model to give more importance to the minority class during training. Without it, the model could trivially predict no incident all the time and ignore the rare spikes.
- **max_iter** increased so the optimizer converges. Logistic regression uses iterative optimization, and with unscaled features or rare events, the default number of iterations might not be enough for the loss function to reach a stable minimum. Increasing **max_iter** ensures the model actually learns meaningful coefficients.
- Statistical features like mean, standard deviation, and trend were added to help the model detect patterns that raw values alone might miss.

The data is split chronologically: 80% train and 20% test. I did not shuffle because order matters for time series.

Evaluation Setup

The model outputs probabilities that an incident will occur. I convert these to alerts using an optimized threshold.

optimal threshold = 0.35

if probability $\geq$ 0.35 $\rightarrow$ predict incident

otherwise $\rightarrow$ predict normal

The threshold was chosen by testing values between 0.1 and 0.9 to maximize the F1 score. I evaluate the model using three standard metrics:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision matters because too many false alerts are annoying. Recall matters because missing incidents is also bad.

Results

With 5000 generated samples, the results were:

Precision: 0.7275

Recall: 0.8562

F1 : 0.7866

These numbers are quite okay. The model catches 86% of incidents while maintaining 73% precision, meaning most alerts are correct. The F1 score of 0.79 shows a good balance between catching incidents and avoiding false alarms.